OBJEKTNO-ORIJENTISANO PROGRAMIRANJE · SMJEROVI B, C, D

Klasa kao modul: dvije datoteke, konstruktori i statički članovi

Klasa i program u istoj datoteci

Naša klasa **Racun** i funkcija **main** nalazile su se do sada u jednoj datoteci. Za jednu klasu, to je praktično, ali nosi sljedeće posljedice:

Klasa se ne može ponovo upotrijebiti

Da bi drugi program koristio klasu **Racun**, kod klase Racun bi se prepisivao u svaki program koji je koristi. Kopija koja se prepisuje prestaje da bude ista klasa čim se jedna od kopija izmijeni.

Poslije svake izmjene moramo prevoditi cijeli kod

Izmjena jednog reda u funkciji **main** ponovo prevodi i cijelu klasu Racun. U programu od stotinu klasa to se mjeri minutima.

Iako nam treba samo interfejs, moramo učitati sva tijela metoda

Da bi se vidjelo šta je sa računom moguće raditi, mora se pročitati i svih pet **tijela metoda te klase**. Interfejs postoji, ali nije izdvojen.

Rješenje

Podjela na fajlove i *razdvojeno prevođenje*: deklaracije u zaglavlju (.h fajl), definicije u izvornoj datoteci (.cpp).

Deklaracija i definicija metode klase

Deklaracija — najava

Ime, tip i ulazni parametri, bez tijela metode. **Prevodiocu** je dovoljna deklaracija metode da bi provjerio svaki **poziv** te metode

```
void uplata(double iznos);
```

Definicija — kako je implementirana metoda

Deklaracija + tijelo. U cijelom programu smije postojati **tačno jednom**; o tome se stara **linker**, a ne prevodilac.

```
void Racun::uplata(double iznos)
{
   if (iznos > 0)
      stanje += iznos;
}
```

Interfejs klase je skup **deklaracija metoda**.

Implementacija klase je skup **definicija njenih metoda**.

Dobra praksa je da **interfejs** i **implementaciju** razdvojimo na **dvije datoteke**.

Datoteka Racun.h — interfejs

```
#ifndef RACUN_H
#define RACUN_H

#include <string>

class Racun {
private:
    const std::string broj;
    std::string vlasnik;
    double stanje;
    static int brojRacuna;   // ukupan broj računa u sistemu

public:
    Racun(std::string broj, std::string ime, double pocetno = 0.0);

    void uplata(double iznos);
    bool isplata(double iznos);
    void prikazi() const;

    double getStanje() const { return stanje; }
    static int getBrojRacuna() { return brojRacuna; }
};

#endif
```

Šta ide u header fajl (.h) ili zaglavlje

Deklaracija klase sa svim atributima i **potpisima** metoda. Kratka tijela, poput getera, smiju ostati u header fajlu.

const std::string broj — broj računa se zadaje pri otvaranju i nikada se ne mijenja.

static int brojRacuna — statički podatak član. To je podatak koji pripada klasi, a ne pojedinom računu.

Ime .h datoteke i ime klase ne moraju biti isti, ali je praksa da jesu.

Datoteka Racun.cpp — implementacija

```
#include "Racun.h"
#include <iostream>
int Racun::brojRacuna = 0;
Racun::Racun(std::string broj, std::string ime, double pocetno)
   : broj(broj), vlasnik(ime), stanje(pocetno > 0 ? pocetno : 0)
{
   brojRacuna++;
}
void Racun::uplata(double iznos)
{
   if (iznos > 0)
      stanje += iznos;
}
bool Racun::isplata(double iznos)
{
      if (iznos <= 0 || iznos > getStanje())
         return false;
      stanje -= iznos;
      return true;
}
void Racun::prikazi() const
{
      std::cout << vlasnik << ": " << stanje << "EUR " << broj <<"\n";
}
```

Prvi red

Izvorna datoteka (.cpp fajl ili source file) uključuje (#include) svoje **zaglavlje (.h fajl)**. Bez toga prevodilac ne bi znao ni da klasa **Racun** postoji.

Prefiks Racun::

Bez njega to ne bi bila metoda klase nego globalna funkcija istog imena.

```
Racun::Racun(std::string broj, std::string ime, double pocetno = 0.0)
```

Podrazumijevana vrijednost argumenta (*pocetno*)
pise se **samo u zaglavlju**;
Ako se ponovi i u .cpp fajlu, prevodilac prijavljuje gresku:

```
error: default argument given for
parameter 3 of 'Racun::Racun(...)'
```

Operator :: i oblast važenja imena

metoda klase Racun

```
void Racun::prikazi() const
{
        std::cout << vlasnik << ": " << stanje << "EUR " << broj <<"\n";
}
```

globalna funkcija istog imena

```
void prikazi()
{
    // nema pristup atributima nijednog racuna
}
```

Primijećujemo 3 različite upotrebe operatora ::

Racun::prikazi — metoda klase.
Racun::brojRacuna — statički atribut.
std::string — ime iz prostora imena standardne biblioteke.

U svim slučajevima operator znači: ime koje slijedi traži se unutar oblasti koja mu prethodi.

Od 3 datoteke pravimo jedan program

main.cpp

```
#include <iostream>
#include "Racun.h"

int main()
{
    Racun a("501-001", "Marko Markovic", 1000);
    Racun b("501-002", "Ana Anic");

    a.uplata(250);
    b.uplata(500);

    a.prikazi();
    b.prikazi();

    std::cout << "Otvoreno racuna: "
              << Racun::getBrojRacuna() << "\n";
}
```

svaka .cpp datoteka prevodi se zasebno i nastaju dvije objektne datoteke, koje onda linker spaja u jedan exe fajl koji mi ovdje nazivamo program

```
g++ -c Racun.cpp -o Racun.o
g++ -c main.cpp  -o main.o
g++ Racun.o main.o -o program
```

izlaz programa

```
501-001  Marko Markovic: 1250 EUR
501-002  Ana Anic: 500 EUR
Otvoreno racuna: 2
```

Ova greška dobija se **od linkera** kada se metoda ***prikazi*** deklariše u zaglavlju a njeno tijelo se ne napiše u .cpp fajlu: prevodilac nije prijavio grešku jer je vidio deklaraciju, a linker traži tijelo i ne nalazi ga pa prijavljuje grešku

```
/usr/bin/ld: undefined reference to
'Racun::prikazi() const'
```

2.1 · PREPROCESOR

Šta radi preprocesor

Prije prevođenja, izvorni kod obrađuje poseban program koji se zove ***preprocesor***. Njegov ulaz je **tekst** i njegov izlaz je **tekst**; Preprocesorske direktive počinju znakom # i nisu naredbe jezika.

Direktiva #include znači da preprocesor na to mjesto upisuje **cijeli sadržaj datoteke koja se uključuje.**

Kada preprocesor naiđe na liniju **#include "Racun.h„ -** efekat je isti kao kada bismo iskopirali tekst datoteke Racun.h na mjesto gdje piše **#include "Racun.h"**)

#include "Racun.h"

Dvostruki navodnici — kompajler traži datoteku Racun.h u folderu projekta.

#include <iostream>

Uglaste zagrade — kompajler traži datoteku iostream.h u sistemskim folderima prevodioca, tamo gdje se nalazi standardna biblioteka.

Koliko je to zapravo linija koda koje analizira prevodilac

Naša datoteka **main.cpp** ima 16 redova. Poslije preprocesiranja prevodilac dobija **36 555** redova, jer su u nju upisana i sva zaglavlja koja je ona uključila. Zato u zaglavlje ne stavljamo ništa što nije neophodno.

Zaštita od višestrukog učitavanja zaglavlja

Kako program raste lako se desi da se neko zaglavlje učita dva puta: dovoljno je da ga uključe dvije datoteke koje se obje uključuju u treću. Klasa bi tada bila deklarisana dva puta i prevodilac bi prijavio grešku. Zato koristimo preprocesorske direktive #ifndef , #define i #endif. Zovu se i zaštitni makroi ili *include guards*

```
#ifndef RACUN_H
#define RACUN_H

// ... deklaracija klase ...

#endif
```

Bez zaštite, javlja se sljedeća greška:

```
error: redefinition of 'class Racun'
```

Kako radi zaštita od višestrukog učitavanja zaglavlja

Pri prvom učitavanju ime **RACUN_H** nije definisano, pa preprocesor kopira sadržaj i definiše ime RACUN_H. Pri svakom narednom učitavanju ime RACUN_H vec postoji, pa sav sadržaj do **#endif** biva preskočen i ne kopira se.

Kraći zapis

#pragma once u prvom redu zaglavlja radi isto i podržavaju ga svi današnji prevodioci, ali nije dio standarda jezika. Oba zapisa su u upotrebi; važno je da zaglavlje ima jedan od njih.

Pokazivač this

Podsjetimo se: Kada su u našem primjeru globalne funkcije postale metode, nestao je parametar **Racun& r** i svi prefiksi **r.**

```
void Racun::uplata(double iznos)
{
   stanje += iznos;       // prevodilac ovo cita kao this->stanje += iznos;
}
```

Skriveni (implicitni) parametar *this*

Svaka nestatička metoda klase dobija pokazivač **this** na objekat nad kojim je pozvana. Poziv **a.uplata(250)** znači: izvrši metodu nad objektom **a**, pri čemu je **this** jednako **&a**. Atributi se unutar metoda mogu pisati **bez ikakvog prefiksa** ili sa prefiksom **this->**

Upotreba pokazivača this je neophodna kada se ulazni argument metode zove isto kao atribut. Tada **this->stanje** označava atribut, a samo **stanje** označava ulazni argument metode.

```
void Racun::setStanje(double stanje)
{
    this->stanje = stanje;
}
```

U zapisu **stanje(stanje)** ispred zagrada uvijek stoji atribut, a u zagradama argument. Zato inicijalizaciona lista ne traži **this**.

Inicijalizacija

Atributi objekta nastaju prije nego što se izvrši **prvi red** tijela konstruktora. Ono što se piše u tijelu nije nastajanje atributa već setovanje vrijednosti.

konstruktor koji se ne prevodi

```
Racun::Racun(std::string broj, std::string ime,
             double pocetno)
{
   this->broj = broj;
   vlasnik = ime;
   stanje = (pocetno > 0) ? pocetno : 0;
}
```

```
error: no match for 'operator='
(operand types are 'const std::string'
and 'std::string')
```

Kada je atribut numerički tip

Za konstantan atribut brojnog tipa poruka je još jasnija i sama upućuje na rješenje:
error: uninitialized const member — *should be initialized*

Objašnjenje greške

Atribut **broj** je konstantan. U trenutku ulaska u tijelo on već postoji i već ima vrijednost, a konstantnoj vrijednosti se ne može dodijeliti druga. Zato je jedini način za setovanje konstantnog atributa kroz incijalizacionu listu

Inicijalizaciona lista u konstruktoru

```
Racun::Racun(std::string broj, std::string ime, double pocetno): broj(broj),
    vlasnik(ime),
    stanje(pocetno > 0 ? pocetno : 0)
{
  brojRacuna++;
}
```

Obavezna je u tri slučaja

kada je atribut **konstantan**; kada je atribut **referenca**; i kada je atribut objekat klase koja nema konstruktor bez argumenata. U svim ostalim slučajevima je preporuka

Redoslijed kao u deklaraciji

Članovi se inicijalizuju u redu u kojem su **deklarisani u klasi**, bez obzira na red u listi. Zato lista treba da prati deklaraciju.

```
warning: 'Racun::vlasnik' will be
initialized after
  'const std::string Racun::broj'
  [-Wreorder]
```

Više konstruktora iste klase

Npr želimo da se račun otvara sa početnim ulogom ili bez njega.

Klasa smije imati više konstruktora, kao što smije imati i više metoda istog imena.

```
class Racun {
public:
    Racun(std::string broj, std::string ime,
          double pocetno);
    Racun(std::string broj, std::string ime);
    // ...
};

Racun::Racun(std::string broj, std::string ime)
    : broj(broj), vlasnik(ime), stanje(0)
{
    brojRacuna++;
}
```

Generalno pravilo preklapanja metoda, ne samo za konstruktore

Dvije metode **istog imena** moraju se razlikovati po **broju** ili po **tipu** argumenata. Razlika samo u povratnom tipu nije dovoljna, jer prevodilac bira metodu po argumentima poziva.

Kada se koja verzija bira

Odluka se donosi pri prevođenju, prema broju i tipu stvarnih argumenata. Za **Racun b("501-002", "Ana Anic")** biće izabran drugi konstruktor.

Podrazumijevani argumenti

Jedan konstruktor može zamijeniti oba sa prethodnog slajda: argument *pocetno* koji je pri pozivu izostavljen, dobija podrazumijevanu vrijednost navedenu u deklaraciji.

```
class Racun {
public:
    Racun(std::string broj, std::string ime,
          double pocetno = 0.0);
    // ...
};

// u Racun.cpp podrazumijevana vrijednost se NE ponavlja
Racun::Racun(std::string broj, std::string ime,
             double pocetno)
```

Podrazumijevanu vrijednost smiju imati samo **posljednji** argumenti u listi parametara, i ona se piše samo na **jednom** mjestu, u deklaraciji, ne ponavlja se u definiciji tijela metode.

Ne mogu postojati oba takva konstruktora istovremeno

Ako se zadrže oba konstruktora sa prethodnog slajda, a jedan od njih dobije podrazumijevanu vrijednost za argument *pocetno*, poziv sa dva argumenta bi odgovarao i jednom i drugom pa imamo grešku:

```
error: call of overloaded
'Racun(const char [8], const char [9])'
is ambiguous
```

Podrazumijevani konstruktor je konstruktor bez argumenata

Na prošlom času ostalo je otvoreno pitanje: zašto red **Racun c;** prestaje da se prevodi čim klasa dobije sopstveni konstruktor.

```
error: no matching function for call
to 'Racun::Racun()'
```

Prevodilac sam pravi podrazumijevani konstruktor (tzv. ugrađeni) samo za klasu koja nije deklarisala **nijedan** svoj konstruktor. Čim klasa definiše makar jedan svoj konstruktor (sa ili bez argumenata), ugrađeni podrazumijevani konstruktor više ne postoji. Tada, ako želimo da ipak imamo podrazumijevani konstruktor, moramo ga definisati.

```
Racun();          // podrazumijevani konstruktor - deklaracija

Racun::Racun() // podrazumijevani konstruktor - definicija
   : broj("000-000"), vlasnik("nepoznat"), stanje(0)
{
   brojRacuna++;
}
```

Ako nam je potreban konstruktor bez argumenata

Piše se kao i svaki drugi konstruktor, samo bez argumenata, i mora sam dati vrijednost svakom atributu. Za klasu **Racun** on nam nije potreban i nećemo ga zadržati: ne želimo da omogućimo kreiranje računa bez broja i bez vlasnika.

Referenca je drugo ime objekta (kaže se i alias)

```
Racun a("501-001", "Marko Markovic", 1000);
Racun& r = a;        // r je drugo ime za a

r.uplata(250);
a.prikazi();         // 501-001  Marko: 1250 EUR
```

Referenca se **mora** inicijalizovati pri nastanku, poslije toga se **ne može** preusmjeriti na drugi objekat.

Preko reference se objektu pristupa se operatorom **tačka**, kao samom objektu.

Na prošlom času funkcije su primale **Racun& r** i **const Racun& r**. Bez reference (&) objekat bi se pri pozivu **kopirao**, pa bi funkcija mijenjala kopiju (predaja argumenata po vrijednosti)

Oznaka **const Racun&** za ulazni argument znači: funkcija dobija pristup objektu bez kopiranja, ali obećava da ga neće mijenjati.

Pokazivač na objekat i operator ->

```
Racun a("501-001", "Marko Markovic", 1000);
Racun* p = &a;        // p sadrzi adresu objekta a

p->uplata(250);       // ovo je isto kao (*p).uplata(250)
p->prikazi();

p = nullptr;          // pokazivac se smije preusmjeriti, referenca ne
```

Operator ->

Skraćeni zapis za dereferenciranje i pristup: **p->uplata(250)** znači isto što i **(*p).uplata(250)**. Prvi zapis je u upotrebi, drugi se rijetko koristi.

Razlika između pokazivača i reference

Pokazivač se smije preusmjeriti i smije biti **nullptr**, pa bi prije upotrebe bilo dobro provjeriti da nije nullptr. Referenca ne može ni jedno ni drugo i zato je za argumente funkcija bolji izbor referenca.

Statički atribut

Svaki račun ima svoje stanje i svog vlasnika. Njih definišemo kao nestatičke atribute.

Ukupan broj otvorenih računa u programu nije svojstvo nijednog računa, već **svojstvo klase**. Dakle svi primjerci (instance) te klase dijele zajedničku informaciju. U takvim slučajevima koristimo statički atribut. Statički atribut je atribut koji **pripada klasi**, a ne objektu.

```
// Racun.h
class Racun {
private:
   static int brojRacuna;     // deklaracija statičkog atributa
public:
   static int getBrojRacuna() { return brojRacuna; }  // definicija statičke metode
};
-------------------------------------------------------------------------------
// Racun.cpp
int Racun::brojRacuna = 0;     // definicija statičkog atributa, tačno jednom
```

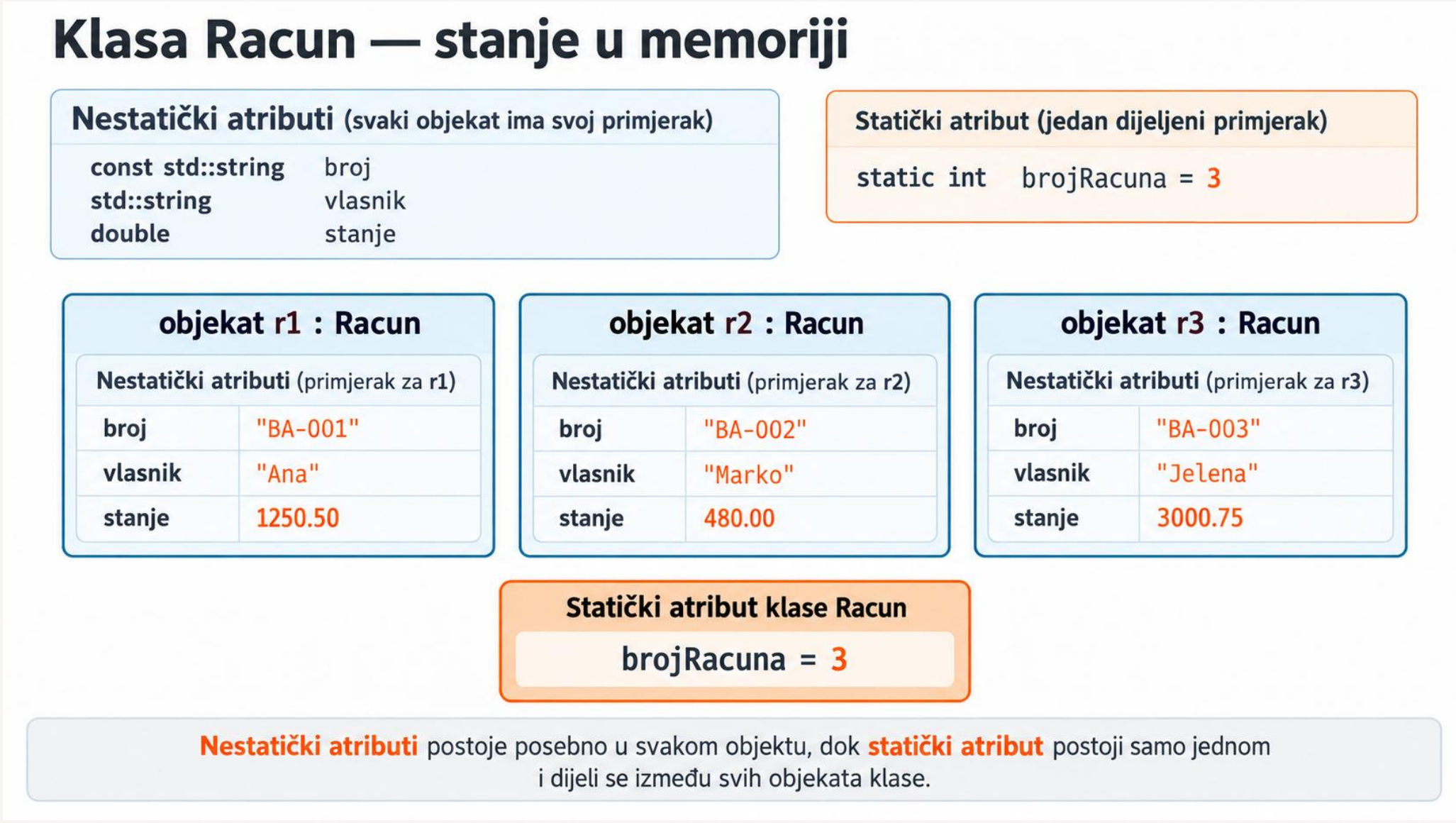
Deklariše se unutar klase.
Definiše se unutar .cpp fajla.

Deklaracija statičkog člana nije i njegova definicija. Mora se definisati posebnom linijom u .cpp fajlu.

Za svaki kreiran objekat klase, u memoriji se formira poseban komplet svih **nestatičkih** atributa te klase, tako da svaki objekat ima sopstvenu memoriju i u toj memoriji se čuvaju vrijednosti njegovih nestatičkih atributa.
Kada je atribut **statički**, u memoriji postoji tačno jedan njegov primjerak za cijelu klasu i on postoji i kada nije napravljen nijedan objekat. Statički atribut je dijeljen između svih objekata te klase.

Svaki pristup statičkom atributu iz bilo kojeg objekta znači i pristup **istoj oblasti u memoriji.**
Može mu se pristupiti i bez navođenja konkrentog objekta, preko imena klase: **Racun::brojRacuna**. Pravila privatnosti važe kao i za ostale članove, pa ako je privatni, izvan klase se čita samo kroz javnu metodu.

Statički atribut



Statičke metode

Navodi se ključna riječ *static* ispred deklaracije metode.

Statičke metode ne pripadaju nijednom objektu – mogu se pozvati iako nije kreiran nijedan objekat klase.

statička metoda ne može pristupati nestatičkim atributima (u ovom slučaju *stanje*)
statička metoda pristupa statičkim atributima klase bez ograničenja (u ovom slučaju brojRacuna)

```
static int getUkupanBrojRacuna()
{
    return brojRacuna + stanje;   // greska jer static metoda ne može pristupiti stanju
}
```

```
error: invalid use of member
'Racun::stanje' in static member
function
```

Statička metoda poziva se bez objekta, kao **Racun::getBrojRacuna()**. Statičke metode nemaju pristup implicitnom pokazivaču **this.**

Statičke metode se koriste onda kada objekat nije potreban: brojači, zajednička podešavanja itd

Pristup statičkim metodama je sličan kao i pristup statičkim atributima. Navodi se ime klase i operator :: ispred imena funkcije, npr. **Racun::getBrojRacuna()**

Može se pristupiti i preko objekata, pomoću operatora pristupa (.i ->)

Važe ista pravila za vidljivost kao i za druge metode

Inline metode

Kratke funkcije poput gettera se isplati definisati u zaglavlju (brže izvršavanje).

Ako želimo da **definišemo** metodu (napišemo njeno tijelo) **unutar** header fajla, onda će se njeno tijelo učitati u sve fajlove koji uključuju taj header fajl (sa #include). U tom slučaju će linker javiti grešku *multiple definition*. Da bismo to izbjegli, takve metode definišemo kao *inline*.

Sve metode čije je tijelo **unutar definicije klase** su **implicitno inline**. Sve metode koje definišemo **u header fajlu** ali **izvan definicije klase**, moramo eksplicitno definisati kao *inline*.

Oznaka *inline* ispred metode čije je tijelo definisano **unutar klase** je dozvoljena ali suvišna; inline definicija je potrebna samo kada tijelo stoji **u header fajlu izvan klase**.

```
class Racun {
public:
    double getStanje() const { return stanje; }
    void prikazi() const;
};
```

Tijelo metode u klasi znači implicitno da je metoda inline

Metoda definisana unutar deklaracije klase implicitno je **inline**. Zato **getStanje** smije imati tijelo u zaglavlju, a **prikazi** ga ima u **Racun.cpp**.

Ako bi deklarisali metodu bez *inline*

Metoda sa tijelom u zaglavlju (ali van klase) ulazi u svaku datoteku koja to zaglavlje uključi, pa linker nalazi više definicija istog imena i prijavljuje grešku multiple definition.

```
/usr/bin/ld: multiple definition
of ’getStanje()'
```

Inline metode

Racun.h

```
#ifndef RACUN_H
#define RACUN_H


#include <string>


class Racun {
private:
   const std::string broj;
   std::string vlasnik;
   double stanje;
   static int brojRacuna;

public:
   Racun(std::string broj, std::string ime,
                 double pocetno = 0.0);


   void uplata(double iznos);
   bool isplata(double iznos);
   void prikazi() const;

   double getStanje() const { return stanje; }
   static int getBrojRacuna() { return brojRacuna; }
};

#endif
```

OK jer je getStanje implicitno inline

Racun.h

```
#ifndef RACUN_H
#define RACUN_H

#include <string>

class Racun {
private:
   const std::string broj;
   std::string vlasnik;
   double stanje;
   static int brojRacuna;

public:
   Racun(std::string broj, std::string ime,
                 double pocetno = 0.0);


   void uplata(double iznos);
   bool isplata(double iznos);
   void prikazi() const;

   double getStanje() const;
   static int getBrojRacuna() { return brojRacuna; }
};

double Racun::getStanje() const { return stanje; }
#endif
```

Greška jer getStanje nije deklarisana kao inline
/usr/bin/ld: multiple definition
of 'getStanje()'

Racun.h

```
#ifndef RACUN_H
#define RACUN_H

#include <string>

class Racun {
private:
   const std::string broj;
   std::string vlasnik;
   double stanje;
   static int brojRacuna;

public:
   Racun(std::string broj, std::string ime,
                 double pocetno = 0.0);


   void uplata(double iznos);
   bool isplata(double iznos);
   void prikazi() const;

   double getStanje() const;
   static int getBrojRacuna() { return brojRacuna; }
};

inline double Racun::getStanje() const { return stanje; }

#endif
```

OK jer je getStanje eksplicitno deklarisana kao inline